Team Project Final Report

CSCI 4435/5435 Spring 2026

Lucas Montoya and Cassandra Palmer

Part I. Midterm Results

1) Problem

Our main goal is to detect anomalous/malicious network behavior from traffic records. The motivation for this project is how traditional rule-based systems often miss novel attacks; anomaly detection can flag unusual behavior sooner.

2) NLP Task

Our basic task is to perform binary sequence classification to determine between benign activity or a genuine attack. We plan to use the Transformer sequence classification from HuggingFace on the CIC-IDS2017 dataset from Kagglehub (dhoogla/cicds2017) [1]. This dataset contains network flow records in a CSV row format which we will serialize into a text string by joining all 77 feature values with spaces (e.g. "0.5 TCP 1200 0..."). The specifications are as follows:

- 2,313,810 records
- 77 features per record
- 15 original labels
- 2 binary labels (benign/attack)
- 7,713,007 vocabulary words
- 1,977,318 Benign and 336,492 Attack records (85% benign)

3) Progress

For text vectorization we have utilized TF-IDF (TfidfVectorizer, max 10,000 features), and for classification we use simple Logistic Regression. Results on a 100,000 sample with 80/20 split are as follows:

- Accuracy: 98%
- Attack F1: 0.94, Precision: 0.97, Recall: 0.92
- Benign F1: 0.99, Precision: 0.99, Recall: 0.99
- Training time: 0.18 seconds

4) Challenges

Our primary challenges include the extremely large vocabulary size (7.7 million tokens), severe class imbalance (85% benign), and training being done on a 100K sample for 2.3 million records.

5) Schedule

Our plan moving forward is to:

1. Train a Transformer model (DistilBERT via HuggingFace) on the serialized text
2. Compare Transformer results vs. baseline
3. Evaluate both using precision, recall, F1
4. Complete final write-up and code cleanup

Part II. Our Process

1 Step 1 — Strengthen the baseline

To improve our results, we implemented a class_weight="balanced" to the LogisticRegression, and added a confusion matrix output. This is done because our Midterm accuracy was inflated by class imbalance; as such, balanced weights and per-class metrics will provide a fairer picture.

1.1 Results (100K sample, 80/20 split):

- Accuracy: 0.9831
- Macro-F1: 0.9675
- Attack: precision 0.9095, recall 0.9834, F1 0.9450
- Benign: precision 0.9971, recall 0.9831, F1 0.9901
- Training time: 0.21 s
- Confusion matrix (rows = true, cols = pred; order Benign / Attack): [[16769 288] ← 288 benign flagged as attacks (false alarms) [49 2894]] ← 49 attacks missed (false negatives)

1.2 Comparison to midterm baseline:

Metric	Midterm	Step 1	Change
Accuracy	0.98	0.9831	About the same
Attack Precision	0.97	0.9095	Less (more false alarms)
Attack Recall	0.92	0.9834	More (catches more attacks)
Attack F1	0.94	0.9450	About the same

1.3 Step 1 Conclusion:

Balanced class weights didn't raise overall accuracy — they shifted the model toward catching more attacks at the cost of more false positives. For an IDS this is the correct tradeoff: a missed attack is more costly than a false alarm an analyst can dismiss.

2 Step 2 — Reduce Vocabulary

This can be accomplished via numeric bucketing. This involves bucketing numeric features into quantile bins before serializing, and rerunning logistic regression on the bucketed text. This end goal is to retain useful feature information while cutting down on the size of our vocabulary.

Our first attempt collapsed the vocabulary into only 5 tokens, since every column shared the same bucket labels (0, 1, 2, 3, 4). These collided with TF-IDF across 77 columns, and the model could not differentiate which feature each bucket came from. With only 5 tokens, our accuracy was 0.7846, our Macro-F1 was 0.6895, and Attack F1 was 0.5177. Less than ideal. Our fix was to keep column identity in each bucket token. For instance, instead of

"3", the token become "c5_b3", or "column 5, bucket 3". Then, TF-IDF learns the weights per-column-per-bucket.

2.1 Results (fixed bucketing, 100K sample, 80/20 split):

- Bucketed vocab size: 242 (vs 7,713,007 midterm, about 32,000 times smaller)
- Accuracy: 0.9746
- Macro-F1: 0.9518
- Attack: precision 0.8686, recall 0.9749, F1 0.9187
- Benign: precision 0.9956, recall 0.9746, F1 0.9849
- Training time: 0.36 s
- Confusion matrix (rows = true, cols = pred; order Benign / Attack): [[16623 434] ← 434 false alarms [74 2869]] ← 74 missed attacks

2.2 Step 1 and Step 2 comparison

Metric	**Step 1 (Balanced LogReg)**	**Step 2 (Bucketed)**
Vocab Size	7,713,007	242
Accuracy	0.9831	0.9746
Attack F1	0.9450	0.9197
Missed Attacks (FN)	49	74
False Alarms (FP)	288	434

2.3 Step 2 Conclusion:

The fixed bucketing trades roughly one percentage point of accuracy for a vocabulary 32,000× smaller and a model that no longer depends on exact numeric values. This makes the model far more compact and generalizes better to traffic with values it never saw during training.

3 Step 3 — Tree-based Baseline

We want our model to be trained with RandomForest directly on the numeric columns with a stratified 80/20 split, because tree-based models avoid information loss associated with tokenizing or bucketing due to splitting directly on continuous features. They also capture nonlinear feature interactions that Logistic Regression simply cannot. This serves as the "non-textual" reference point we measure text-based approaches against.

3.1 Results (100K sample, 80/20 split):

- Accuracy: 0.9981
- Macro-F1: 0.9962
- Attack: precision 0.9948, recall 0.9921, F1 0.9935
- Benign: precision 0.9987, recall 0.9991, F1 0.9989
- Test split support: 2908 Attack / 17092 Benign (stratified — slightly different from Steps 1–2 which used a non-stratified split)

3.2 Top 10 feature importances:

1. Bwd Packet Length Mean — 0.0691
2. Packet Length Variance — 0.0661
3. Bwd Packet Length Max — 0.0661
4. Avg Bwd Segment Size — 0.0607
5. Bwd Packet Length Std — 0.0599
6. Packet Length Std — 0.0552
7. Avg Packet Size — 0.0438
8. Bwd Packets Length Total — 0.0435
9. Bwd Packets/s — 0.0362
10. Bwd Packet Length Min — 0.0341

3.3 Step 3 Conclusion:

RandomForest is the strongest model so far. This raises a fair question — *if numeric features work this well, why use NLP at all?* The honest answer: the project's NLP framing (treating each network flow as a serialized text sequence) is what enables transformer methods like DistilBERT in Step 4. RF establishes the ceiling a non-text model can reach on this data, which is the right comparison point for a text-based pipeline.

Note: Steps 1 and 2 used a non-stratified split. This can be fixed by re-running them with stratify=y.

4 Step 4 — Transformer (DistilBERT)

Here we fine-tune DistilBERT on the serialized text with a sample of 50K rows, natural class distribution (about 85% benign and 15% attack), and an 80/20 split (40K training rows and 10k testing rows). This was performed on an NMSU CS GPU with 2 epochs and max_length=128. We do this to address the midterm feedback of trying more methods, including transformer-based methods. We re-ran it on the natural distribution instead of an artificially balanced subset, so the results are directly comparable to Steps 1 through 3.

4.1 Results (10K test set, natural distribution):

- Accuracy: 0.9951
- Macro-F1: 0.9901
- Attack: precision 0.9923, recall 0.9740, F1 0.9831
- Benign: precision 0.9956, recall 0.9987, F1 0.9971
- Training time: 203.78 s (about 24.5 it/s on GPU)
- Confusion matrix (rows = true, cols = pred; order Benign / Attack): [[8527 11] ← 11 false alarms [38 1424]] ← 38 missed attacks
- Total errors: 49 / 10,000
- Test split support: 8538 Benign / 1462 Attack (matches the natural 85/15 ratio)

4.2 Step 4 Conclusion:

DistilBERT made only 49 errors out of 10,000 and was easily the strongest text-based model — beating both Logistic Regression baselines in Steps 1 and 2. It did not, however, surpass RandomForest on raw numeric features in Step 3. The training cost is real: about 204 s on GPU vs sub-second LogReg and a few seconds for RF. The transformer demonstrates that contextual representations of serialized network features work strongly, but tree-based numeric models remain a competitive non-NLP alternative for this task.

Note: Attack recall (0.9740) is the lowest among the models, reflecting the class imbalance — without class weighting in BERT's loss, the model leans slightly toward predicting Benign. A weighted loss or oversampling would likely close this gap.

5 Step 5 — Final Comparison

All models were trained on a 100K sample (BERT on 50K) of CIC-IDS2017 with and 80/20 split. RandomForest used a stratified split, while the others used a non-stratified random split; supports differ slightly.

Model	**Input**	**Accuracy**	**Macro-F1**	**Attack F1**	**Attack Recall**	**Training Time**
LogReg (midterm)	TF-IDF raw text	0.9800	0.9700	0.9400	0.9200	0.27 s
LogReg + Balanced	TF-IDF raw text	0.9831	0.9675	0.9450	0.9834	0.21 s
LogReg + Bucketed Vocab	TF-IDF bucketed text	0.9746	0.9518	0.9187	0.9749	0.36 s
RandomForest	Raw numeric, no text	0.9981	0.9962	0.9935	0.9921	1.29 s
DistilBERT	Serialized text	0.9951	0.9901	0.9831	0.9740	203.78 s

The **best model overall** was RandomForest based on raw numeric features such as Accuracy, Macro-F1, and Attack F1. The **best NLP model** was DistilBERT, which beat both TF-IDF + LogReg variants on Attack F1 with only 49 errors across 10K samples. We believe that RandomForest won because of how trees split directly on continuous features such that no information is lost to tokenization or bucketing. Additionally, they model nonlinear features interactions that LogReg cannot. The reason DistilBERT is the right NLP answer is that NLP framing for this project requires a text-based pipeline. Among the text models, DistilBERT's contextual representations clearly beat TF-IDF + LogReg, justifying the cost of a transformer for this kind of serialized data.

5.1 Error Analysis:

Model	Test Size	Missed Attacks (FN)	False Alarms (FP)	FN Rate	FP Rate
LogReg + Balanced	20,000	49 / 2,943	288 / 17,057	1.7%	1.7%
LogReg + Bucketed	20,000	74 / 2,943	434 / 17,057	2.5%	2.5%
RandomForest	20,000	about 23 / 2,908	about 22 / 17,092	0.8%	0.1%
DistilBERT	10,000	38 / 1,462	11 / 8,538	2.6%	0.1%

DistilBERT yielded the fewest false alarms at only 11, since its contextual representation is very precise about what counts as benign. Both DistilBERT and Bucketed LogReg have higher FN rates than Step 1; without class weighting, they tilt toward predicting the majority class (Benign). RandomForest dominates on both axes thanks to direct numeric splits and class-balanced weights.

Note: To produce a per-attack-category breakdown, rerun rf_numeric.py with the original 15-label Label column kept alongside binary_label, then group misclassified rows by original label. Infiltration and rare web-attack subtypes will likely be the hardest to find, because they have the fewest training examples.

6 Limitations / Future Work:

- All experiments used a 100K sample (50K for DistilBERT) of the 2.3M-row dataset. Training on the full dataset would likely improve all models marginally and is the obvious next scaling step.
- DistilBERT did not use class-weighted loss — its 0.974 Attack recall is the lowest among final models. Adding a weighted cross-entropy loss or oversampling the Attack class would likely close the gap.
- A first-attempt naive bucketing collapsed the vocab to 5 tokens because column identity was lost; this is documented as a learned mistake.
- RandomForest (Step 3) used a stratified split; the LogReg variants used a non-stratified split. Supports differ slightly (2,908 vs 2,943 Attack). Per-class F1 metrics remain comparable but the difference is footnoted in Step 3.
- All evaluation is binary (Benign vs Attack). A multi-class evaluation using the original 15 labels would reveal which attack families are hardest to detect.

Part III. Final Report

7 Problem & Motivation:

We want to detect anomalous or malicious activity in network flow records. Traditional intrusion detection systems rely on signature- and rule-based detection, which require known attack patterns and miss novel attacks until rules are written for them. A learned classifier that flags unusual flows can surface previously unseen attack behavior earlier, giving security analysts a faster signal than rule-only systems provide. We frame this detection task as binary classification (Benign vs Attack) over the CIC-IDS2017 dataset, with our text-based variants treating each network flow as a serialized "sentence" so it can be fed to NLP models including a transformer.

8 NLP Task:

Our NLP task is binary sequence classification: given a serialized network-flow record as text, predict Benign or Attack. Out dataset is CIC-IDS2017, a public network intrusion dataset originally released by the Canadian Institute for Cybersecurity. We obtained the parquet-formatted version dhoogla/cicids2017 from Kaggle via kagglehub. The dataset captures five days of simulated benign and attack traffic (Monday benign baseline; Botnet, Brute-force, DDoS, DoS, Infiltration, Portscan, Web Attacks across the other days). Each row is a network flow record described by 77 numeric features (packet counts, byte counts, inter-arrival times, flag counts, etc.). To turn a row into text, we serialize each row by joining

all 77 feature values with spaces, e.g. "0.5 TCP 1200 0 ...". This gives every row a single sentence-like representation that NLP models can tokenize. The stats of the dataset are as follows:

- Total records: 2,313,810
- Features per record: 77
- Original labels: 15 (Benign + 14 attack subtypes)
- Binary labels used here: 2 (Benign / Attack)
- Class distribution: 1,977,318 Benign / 336,492 Attack (about 85% Benign)
- Vocabulary (raw serialization): 7,713,007 unique tokens (every distinct numeric value becomes its own token)
- Vocabulary (after column-prefixed quantile bucketing, Step 2): 242

The way we processed the data is as follows:

1. Load all per-day parquet files and concatenate into one DataFrame.
2. Add a binary label column (Benign or Attack).
3. Replace ±inf and NaN numeric values with 0 (data cleaning).
4. Two serialization variants:
    - Raw serialization (Steps 1, 4): join all 77 feature values per row as strings.
    - Bucketed serialization (Step 2): discretize numeric features into 5 quantile bins, prefix each value with its column index ("c5_b3" = column 5, bucket 3) so tokens stay column-specific, then join.
5. Sample 100K rows (50K for DistilBERT) and split 80/20 train/test.
6. Vectorization:
    - LogReg variants: TF-IDF (max_features=10000, token pattern \b\w+\b so underscores in c5_b3 are kept as a single token).
    - DistilBERT: WordPiece tokenizer from distilbert-base-uncased with max_length=128, padding to max length, truncation enabled.
    - RandomForest baseline: no text — feeds the 77 numeric features directly.

Local machine (used for LogReg variants and RandomForest):

- MacBook Pro (Mac17,2)

- Chip: Apple M5
- Cores: 10 (4 Super + 6 Efficiency)
- Memory: 16 GB
- macOS 26.3.1
- Python 3.14.3

CS server (used for DistilBERT only):

- Dell PowerEdge R7525 (kaiju)
- CPU: AMD EPYC 7313 — 2 sockets, 32 cores (16) / 64 threads (32) @ 3.0 GHz
- Memory: 512 GiB
- GPU: NVIDIA A100-PCIE-40GB — 40 GiB, 6912 cores, CUDA 12.0, Compute 8.0
- General-access GPU node (use must be coordinated)

Reproducibility settings (all scripts):

- Random seed: 42
- Sample size: 100K rows for LogReg + RF, 50K for DistilBERT
- Train/test split: 80/20
- rf_numeric.py uses stratify=y; the LogReg variants use a non-stratified split

Which script produced which row in the comparison table:

Comparison Row	Script	Run on	Notes
LogReg (midterm baseline)	main_baseline.py	Local (M5)	Numbers also reported in midterm report.
LogReg + Balanced	main_balanced.py	Local (M5)	Step 1
LogReg + Bucketed	main.py	Local (M5)	Step 2, with column-prefixed quantile bucketing.
RandomForest	rf_numeric.py	Local (M5)	Step 3, raw numeric features.
DistilBERT	distilbert_run.py	CS Server (A100)	Step 4, natural class distribution

For **Implementation and experiment results**, see Step 5 of Part 2.

References

[1] S. . L. D'hooge, "Cic-ids2017," Aug 2022.

[2] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *International Conference on Information Systems Security and Privacy*, 2018.

[3] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "Distilbert, a distilled version of bert: smaller, faster, cheaper and lighter," 2020.

[4] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz, J. Davison, S. Shleifer, P. von Platen, C. Ma, Y. Jernite, J. Plu, C. Xu, T. L. Scao, S. Gugger, M. Drame, Q. Lhoest, and A. M. Rush, "Huggingface's transformers: State-of-the-art natural language processing," 2020.

[5] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, A. Müller, J. Nothman, G. Louppe, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and Édouard Duchesnay, "Scikit-learn: Machine learning in python," 2018.

[6] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, p. 5–32, Oct. 2001.

[7] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," *Inf. Process. Manage.*, vol. 24, p. 513–523, Aug. 1988.